

Introduction of Verilog

Subject: Digital Circuits and Systems



Netaji Subhas University of Technology Delhi
Department of Electronics and Comm. Engg.

Dr. Neeraj Goel
(Assistant Professor)

Table of Contents

- ❖ **Introduction and Components of Verilog Code_ Hardware Language**
- ❖ **Module and End Module Declaration**
- ❖ **Parameter Declaration**
- ❖ **Port Declaration**
- ❖ **Wire/Reg Declaration**
- ❖ **Module Instantiations**
- ❖ **Types of Modeling Style**
- ❖ **Logic - Combinational, Sequential**
- ❖ **Verilog implementation of adder, subtractor, mux-demux, decoders, comparators**
- ❖ **Verilog Implementation of Flip Flops, latches, Registers, Counters**

Introduction

❖ Verilog is a **Hardware Description Language** : Any Digital Design can be described using Verilog.

Use : For Describing Huge Designs (Even Microprocessors!).

❖ A verilog file is given to EDA(Electronic Design Automation) tool to program programmable Logic Devices like Complex Programmable Logic Devices (CPLDs) or Field Programmable Gate Arrays (FPGAs)

❖ General software used: Xilinx, Quartus (Intel), Synopsys VCS

General Verilog Code Components

1. Module Declaration

2. Parameter Declaration

3. Port Declaration

4. Wire/ Reg Declaration

5. Assignment Statements

6. Combinational Logic

7. Always Block

8. If Statements

```
module mydesign #(
  parameter WIDTH = 3,
  parameter FREQ = 100
)(
  input [WIDTH-1 : 0] a,
  input [WIDTH-1 : 0] b,
  output [WIDTH-1 : 0] c,
  output c1,
  output c2,

  input clk
);

wire w_a1, w_a2;
reg r_c1, r_c2;

assign c = a & b;|
assign w_a1 = a[0];
assign w_a2 = a[1]

always @(posedge clk)
begin
  if(w_a1 == 0)
  begin
    r_c1 = w_a2;
  end
  else
    r_c2 = w_a2;
  end

  assign c1 = r_c1;
  assign c2 = r_c2;

endmodule
```

Lower case is often used for signals and variables (e.g., input, output, wire, etc.)
Upper case may be used for constants or macros, often to distinguish them visually

Module Declaration

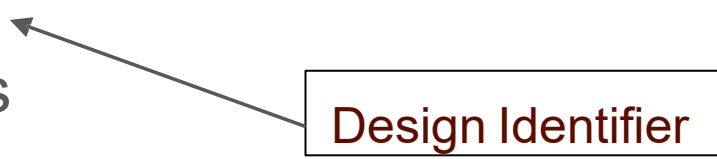
```
module mydesign(
```

```
// Port Declarations
```

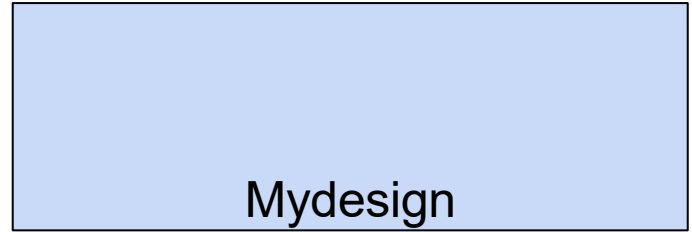
```
);
```

```
// Module Body
```

```
end module
```



Design Identifier



Mydesign

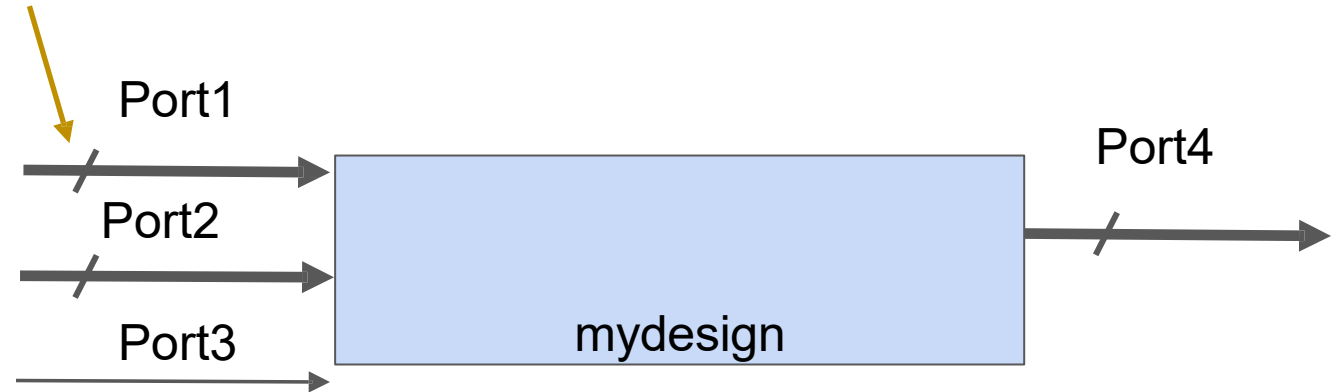
- Marks the beginning and end of the module

Port Declarations

```
module mydesign (  
    input  [1 : 0] port1,  
    input  [1 : 0] port2,  
    input  port3,  
    output [1 : 0] port4 );
```

↑ Port Direction ↑ Port Width ↑ Port Name

2 bit Wide Ports



- Above code declares the input output and inout ports of the module

Wire and Reg Declaration

```
module module_name(
```

```
    input [1 : 0] port1,
```

```
    input [1 : 0] port2,
```

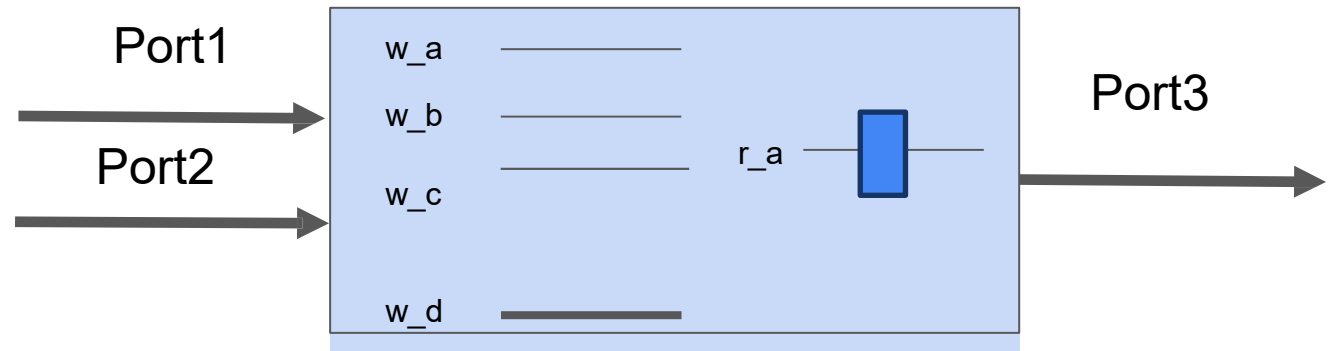
```
    output [1 : 0] port3 );
```

```
    wire w_a, w_b, w_c;
```

```
    wire [1:0] w_d;
```

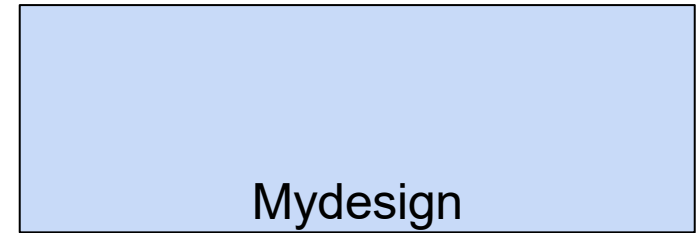
```
    reg r_a;
```

```
endmodule
```



Parameter Declaration

```
module mydesign #(
  parameter parameter_name = value
) (
  // Port declaration
);
endmodule
```

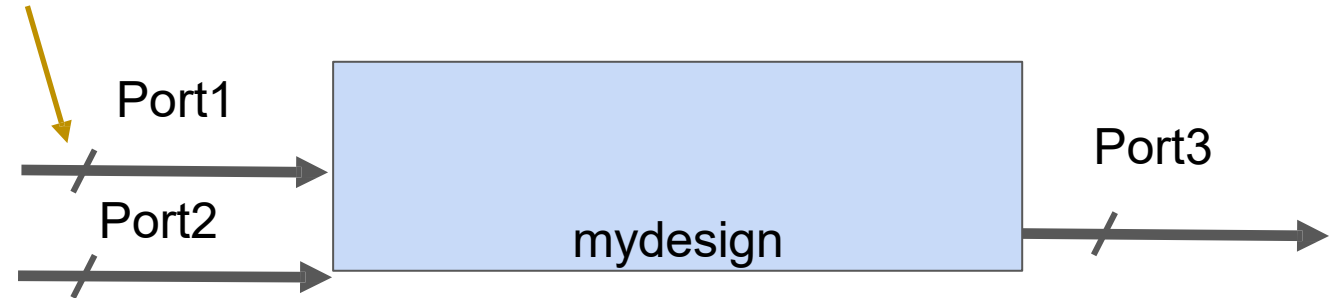


- Parameters are defined for using constants throughout the design.
- Scalability becomes easier when verilog codes are parameterised

Port Declarations with Parameter declaration

```
module mydesign #(  
parameter WIDTH = 32  
)(  
Input [WIDTH-1 : 0] port1,  
input [WIDTH-1 : 0] port2,  
output [WIDTH-1 : 0] port3 );
```

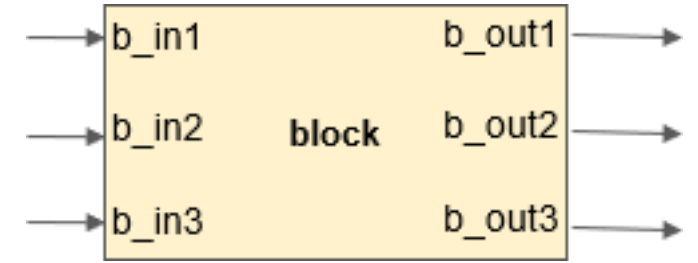
32 bit Wide Ports



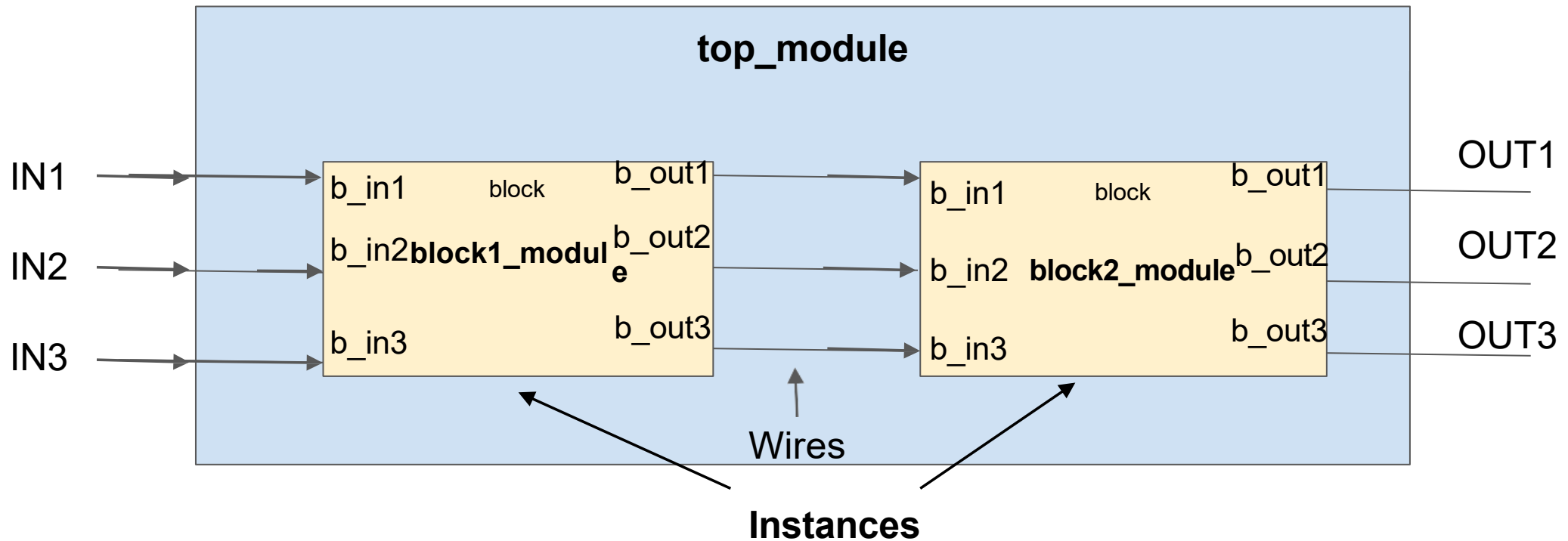
Port Direction Port Width Port Name

- Use symbol # (hash) after module name while declaring the parameters

Module Instantiations



- Various designs follow hierarchies, with top level (Wrapper) module instantiating other blocks of design, integrated together in the wrapper
- The wrapper may also contain other elements of verilog code.



Module Instantiation Example

```
module top_module(  
    input IN1,  
    input IN2,  
    input IN3,  
  
    output OUT1,  
    output OUT2,  
    output OUT3);
```

```
    wire W_1, W_2, W_3;
```

```
    block block1_MODULE (  
        .b_in1(IN1),  
        .b_in2(IN2),  
        .b_in3(IN3),  
        .b_out1(W_1),  
        .b_out2(W_2),  
        .b_out3(W_3)
```

```
    );
```

Block Instantiation

Block identifier

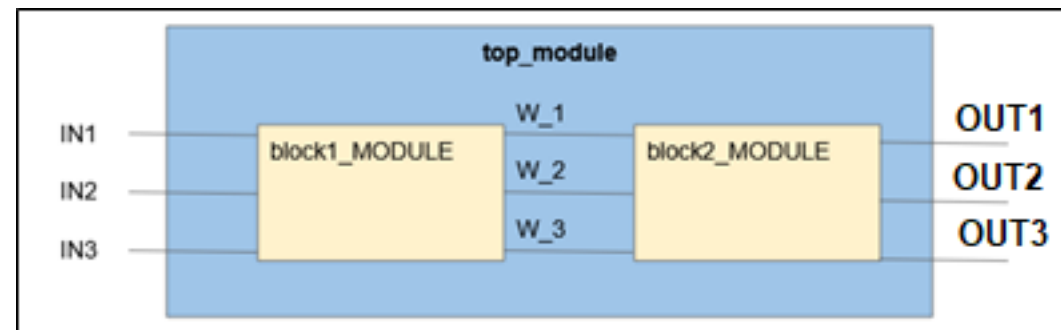
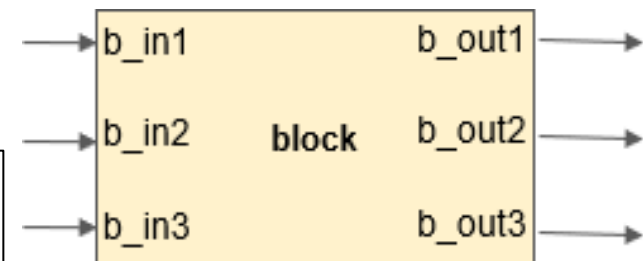
Block instance name/identifier

Block ports

Top Module Wires/Ports

```
        block block2_MODULE (  
            .b_in1(W_1),  
            .b_in2(W_2),  
            .b_in3(W_3),  
            .b_out1(OUT1),  
            .b_out2(OUT2),  
            .b_out3(OUT3)  
        );
```

```
    endmodule
```



Port connection - Name

```
module design_top;
    wire [1:0] a;
    wire      b, c;

    mydesign d0 ( .x (a[0]),      // signal "x" in mydesign should be connected to "a[0]"
                 .y (b),        // signal "y" in mydesign should be connected to "b" in
                 .z (a[1]),
                 .o (c));

endmodule
```

- A better way to connect ports is by explicitly linking ports on both sides using the respective **port name**.
- The dot **.** represents that the port name following the dot belongs to the design.
- The signal name in the module to which the port has to be connected is specified within parentheses.

As the connections are made by name, the order in which they are specified is not important.

Port connection - Ordered list

```
module mydesign ( input  x, y, z,      // x is at position 1, y at 2, x at 3 and
                  output o);         // o is at position 4

endmodule

module tb_top;
    wire [1:0]  a;
    wire        b, c;

    mydesign d0 (a[0], b, a[1], c); // a[0] is at position 1 so it is automatically connected to x
                                   // b is at position 2 so it is automatically connected to y
                                   // a[1] is at position 3 so it is connected to z
                                   // c is at position 4, and hence connection is with o
endmodule
```

- The ports are connected to the design module based on their position in the parameters for module instantiation.
- This port connection style requires us to remember the exact order of ports in the design module (Cumbersome, prone to mistakes)

Combinational Logic : i) Continuous Assignment

Continuous Assignment Statements are used to form connections between two wires / reg/ input/ output entities. Inputs cannot be on the LHS of assign statements.

```
module top_module(  
    input IN1,  
    input IN2,  
    output OUT );  
  
    wire W1;  
  
    assign OUT = IN1;  
    assign OUT = IN2;  
  
endmodule
```



Continuous Assignment Statements

1. Assigning Constant Values

```
module constant_assignment ( output a,  
output [3:0] b );
```

```
assign a = 1'b0;
```

```
assign b = 4'b1010; // 4'b1010 = 4'hA
```

```
endmodule
```

Continuous Assignment Statements

2. Implementing Bitwise Operation Using Assign Statements -

assign out = a___b; // Bitwise operation between “a” and “b” and assigns the result to “out”.

Logical Operation	Operator
and	&
or	
not	!
nand	~&
nor	~
xor	^

Types of Modeling Style

The Verilog HDL modeling language supports three main modeling styles:

- ❖ **Behavioral:** Used for both combinatorial and sequential circuits
- ❖ Verilog code can also be written in a mixed style
- ❖ **Dataflow:** Used to model combinatorial circuits and specifies how data flows between hardware registers and how it is processed
- ❖ **Gate-Level:** Used to model combinatorial circuits
- ❖ **Structural Modeling:** Represents the hardware structure by instantiating modules and connecting them
- ❖ **Verilog** is a language used to model the high-level behavior of analog components and networks

Behavioral Modeling

Definition: Describes how a circuit behaves in terms of its inputs and outputs

Key Points:

- Commonly uses **always** and initial blocks
- High-level abstraction
- **reg** type and **begin**...end syntax are primarily used in behavioral modeling to describe the functionality of the design using high-level constructs
- Uses always blocks, initial blocks
- Easier to understand and implement complex logic

Example:
module behavioral_example(input wire a, b,
output reg y);
always @* begin
y = a & b; // AND operation
end
endmodule

Gate-Level Modeling

❖ Basic Elements:

- Uses basic logic gates such as **AND**, **OR**, **NOT**, **NAND**, **NOR**, **XOR**, and **XNOR**
- Represents each gate as a module

❖ Structural Representation:

- Explicitly describes how **gates are wired** together
- Describes the interconnections between gates explicitly
- Useful for synthesizing circuits directly from the Verilog code

```
Example:  
module gate_level_example(input a,  
    b, output y);  
    wire temp;  
    and (temp, a, b); // AND gate  
    not (y, temp);    // NOT gate  
endmodule
```

❖ Explicit Timing:

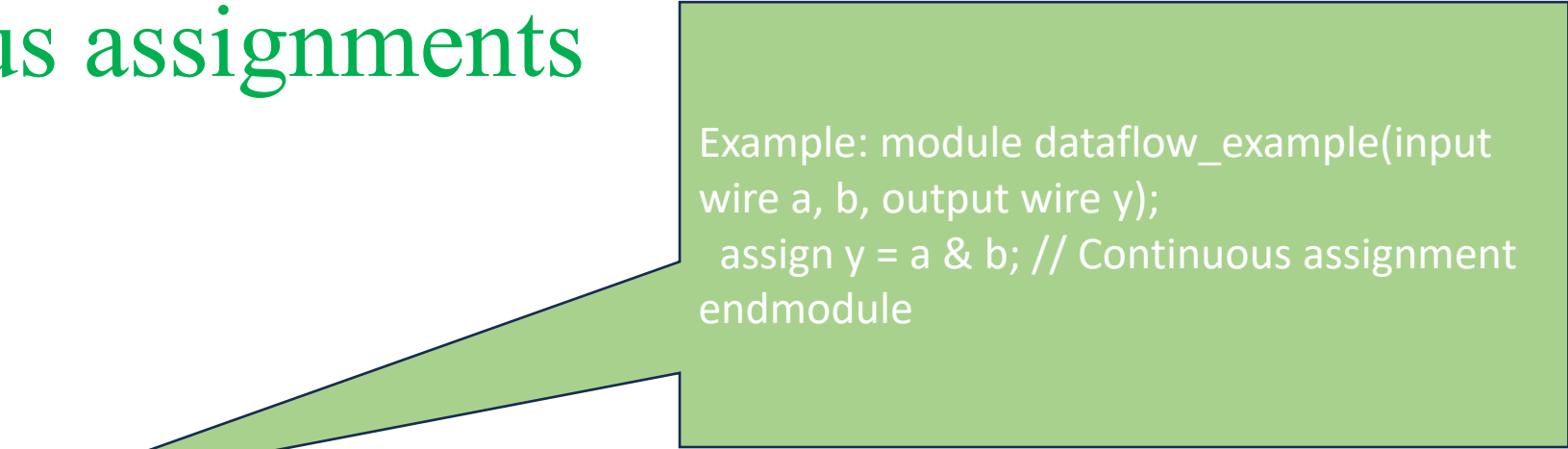
- Timing delays can be added to model real-world behavior more accurately
- Allows simulation of gate propagation delays

Dataflow Modeling

Definition: Describes data transfer and the flow of data using continuous assignments

Key Points:

- ❖ Uses **assign** statements
- ❖ Focuses on the relationship between inputs and outputs
- ❖ Suitable for combinational logic



```
Example: module dataflow_example(input  
wire a, b, output wire y);  
    assign y = a & b; // Continuous assignment  
endmodule
```

Structural Modeling

Definition: Describes how a circuit is built from components

Key Points:

```
Example:  
module structural_example(input wire a, b, output  
wire y);  
    wire temp;  
    and and_gate(temp, a, b); // Instance of AND gate  
    assign y = temp;  
endmodule
```

- Describes how various components are **wired** together.
- Represents the physical connections and hierarchy
- Suitable for large designs

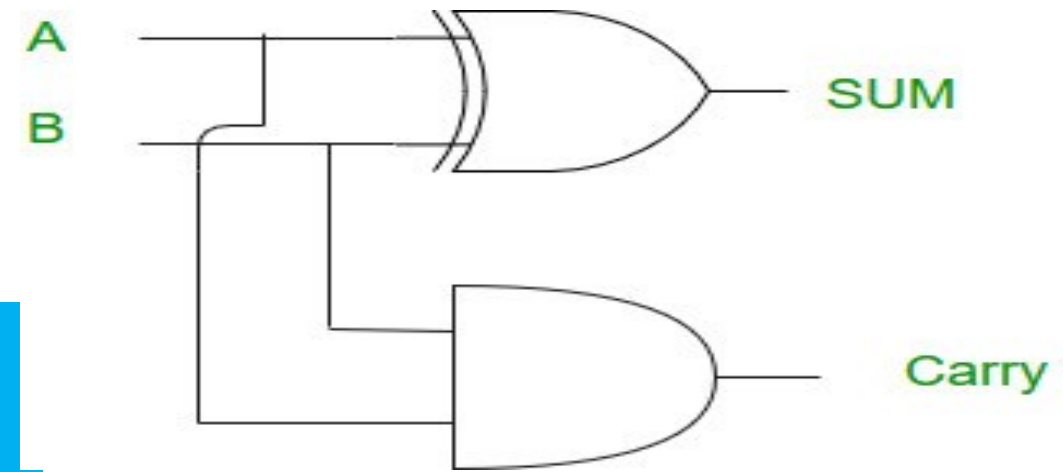
Half Adder in Verilog

(Behavioral Modeling):

```
module half_adder_behavioral(  
    input A,  
    input B,  
    output reg Sum,  
    output reg Carry  
);  
  
always @(*) begin  
    Sum = A ^ B; // XOR for sum  
    Carry = A & B; // AND for carry  
end  
  
endmodule
```

(Dataflow Modeling):

```
module half_adder_dataflow (  
    input A,  
    input B,  
    output Sum,  
    output Carry  
);  
  
assign Sum = A ^ B; // XOR for sum  
assign Carry = A & B; // AND for carry  
  
endmodule
```



(Gate level) Modeling :

```
module half_adder_gate_level(  
    input A,  
    input B,  
    output Sum,  
    output Carry  
);  
  
wire A_and_B;  
wire A_xor_B;  
  
and (Carry, A, B); // AND gate for  
carry  
xor (Sum, A, B); // XOR gate for sum  
  
endmodule
```

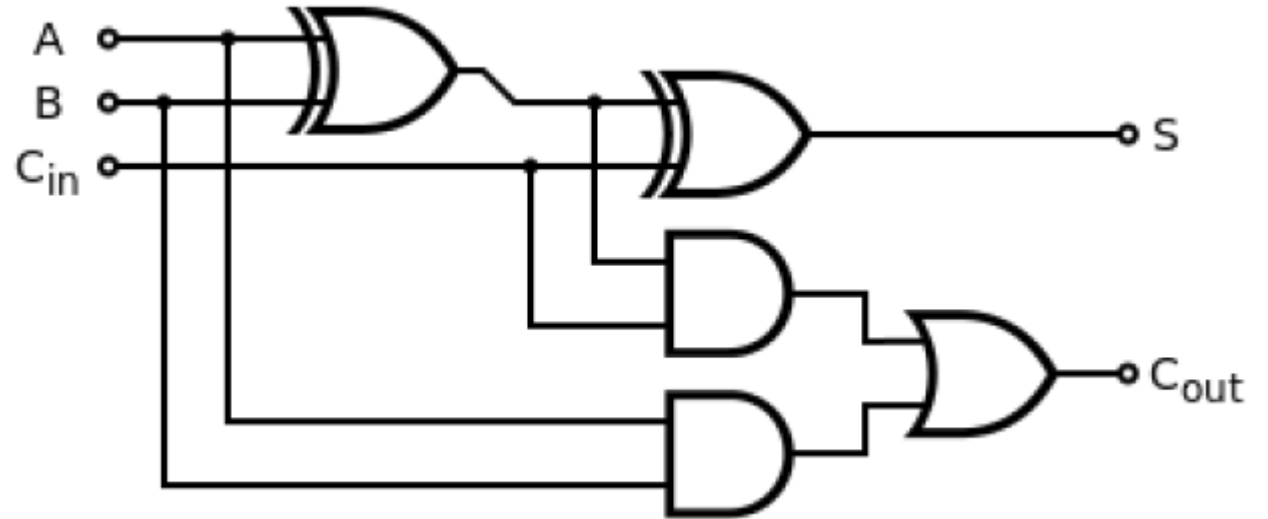
Logical Expression for half adder is:

$S = A \oplus B$; $C = A * B$.

Full Adder in Verilog

•Code Example (Dataflow):

```
module full_adder(  
    input a,  
    input b,  
    input cin,  
    output sum,  
    output cout  
);  
    assign sum = a ^ b ^ cin; // Sum  
    assign cout = (a & b) | (cin & (a ^ b)); //  
Carry out  
endmodule
```

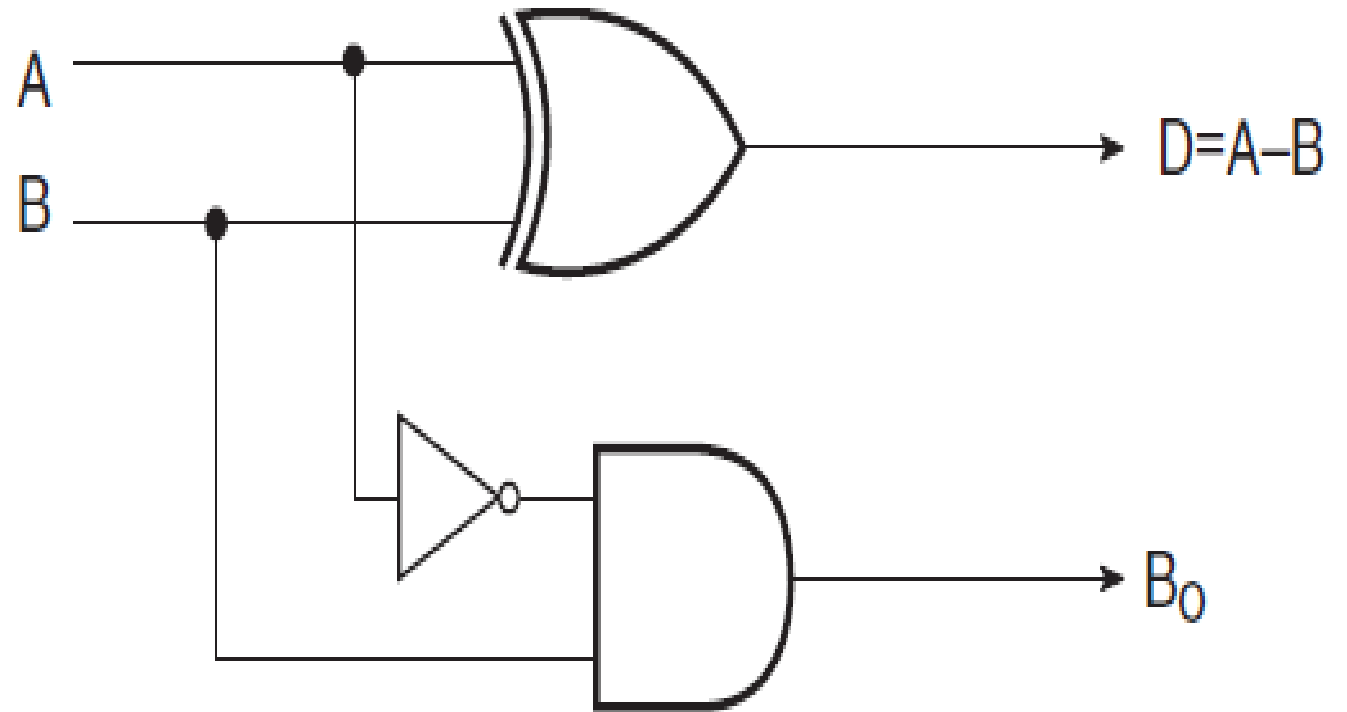


Half Subtractor

The half subtractor takes two inputs, A and B, and produces two outputs: the difference (D) and the borrow (B_out).

•Code Example (Dataflow):

```
module half_subtractor (  
    input A,  
    input B,  
    output D,  
    output B_out  
);  
    assign D = A ^ B;    // Difference is A XOR B  
    assign B_out = ~A & B; // Borrow is NOT A  
    AND B  
endmodule
```

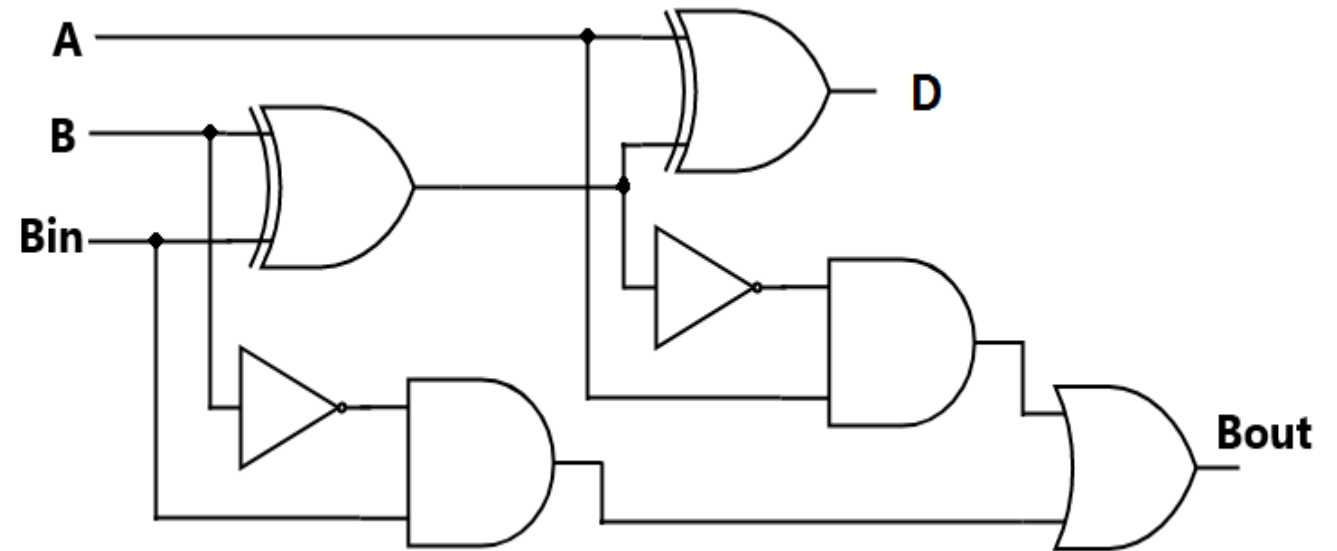


Full Subtractor

- ❖ The full subtractor takes three inputs: A, B, and a borrow input (B_in) from a previous operation
- ❖ It produces a difference (D) and a borrow output (B_out)

•Code Example (Dataflow):

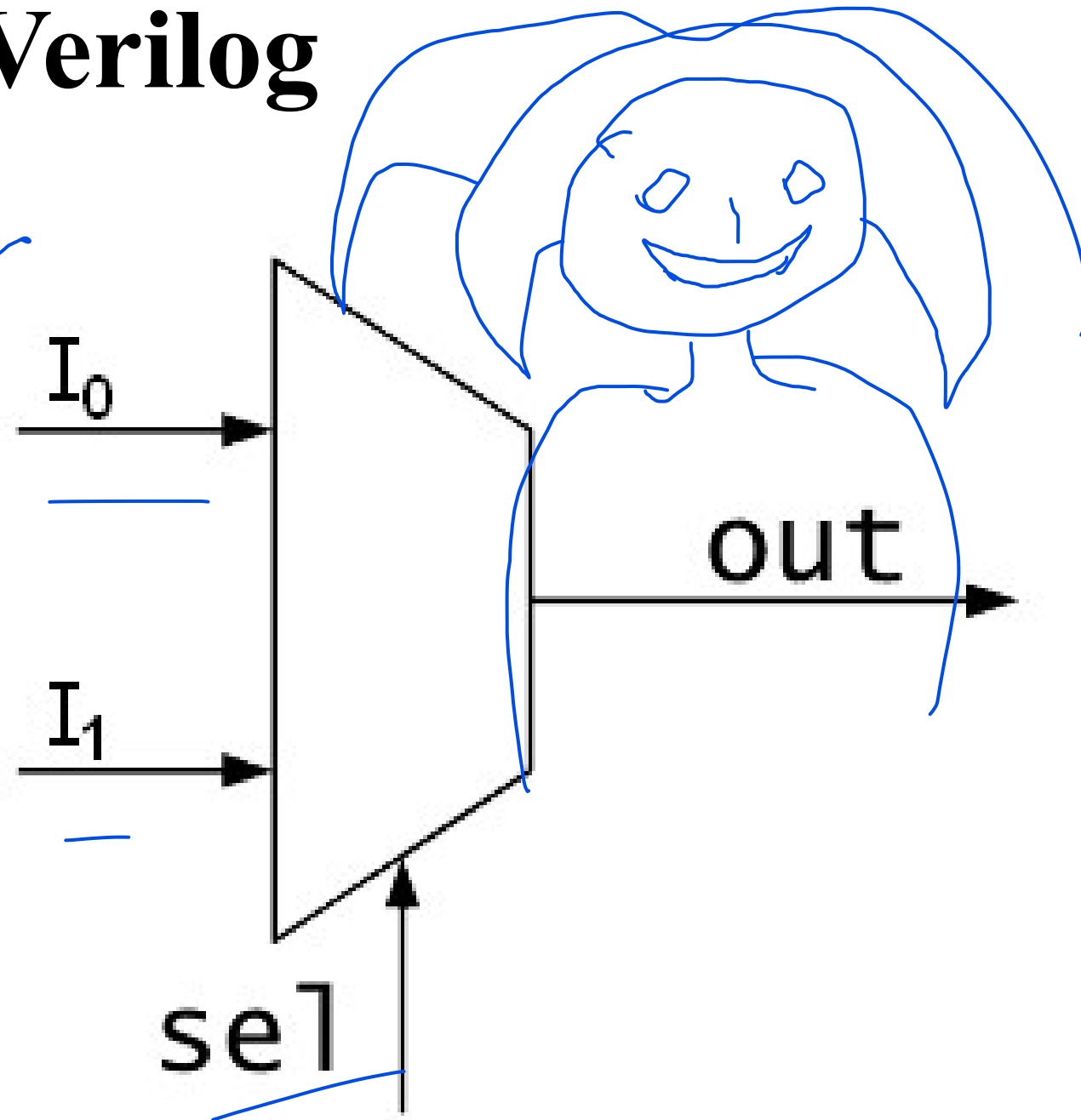
```
module full_subtractor (  
    input A,  
    input B,  
    input B_in,  
    output D,  
    output B_out  
);  
    wire D1, B1, B2;  
  
    // First half subtractor  
    assign D1 = A ^ B;      // Difference of A and B  
    assign B1 = ~A & B;     // Borrow from A and B  
  
    // Second half subtractor  
    assign D = D1 ^ B_in;   // Final difference considering B_in  
    assign B2 = ~D1 & B_in; // Borrow from D1 and B_in  
    assign B_out = B1 | B2; // Final borrow output  
endmodule
```



Multiplexer (Mux) in Verilog

•Code Example (Dataflow):

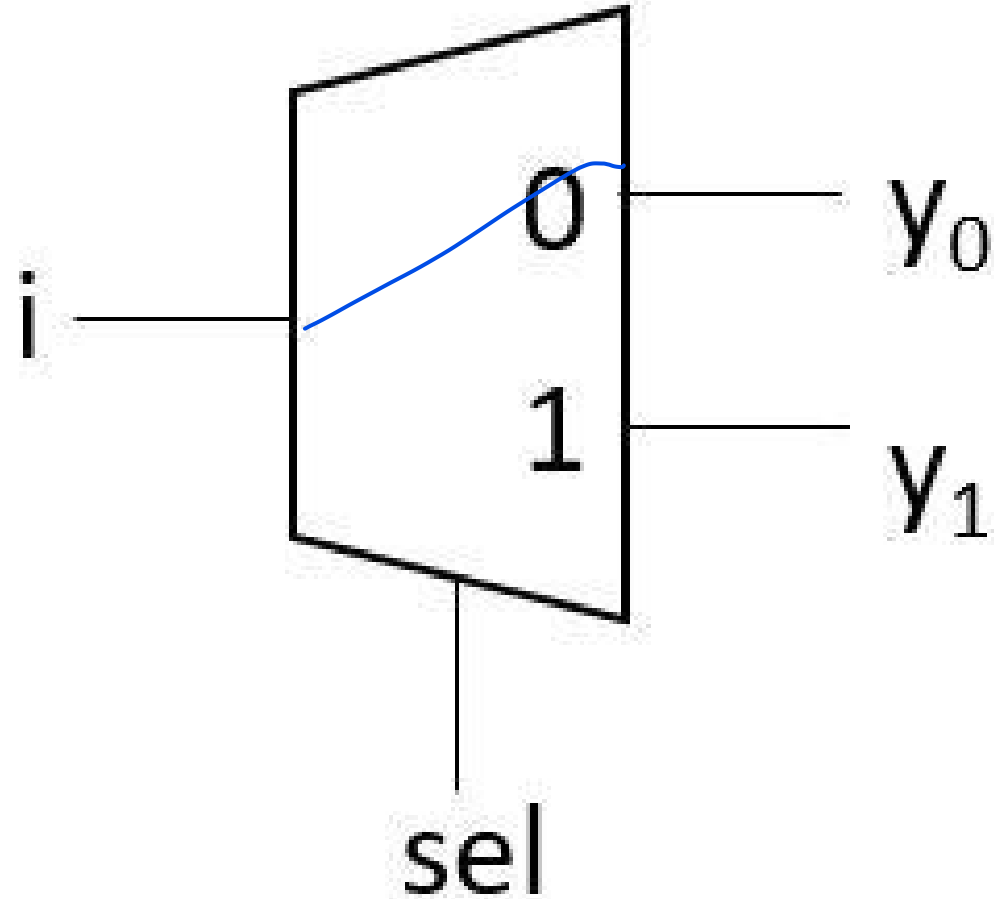
```
module mux2to1(  
    input I0,  
    input I1,  
    input sel,  
    output out  
);  
assign always out = sel ? I1 : I0; // Output based on  
selector  
endmodule
```



Demultiplexer (Demux) in Verilog

Code Example (Behavioral):

```
module demux1to2(  
    input i,  
    input sel,  
    output reg y0,  
    output reg y1  
);  
    always @(*) begin  
        if (sel) begin  
            y0 = 0;  
            y1 = i;  
        end else begin  
            y0 = i;  
            y1 = 0;  
        end  
    end  
end  
endmodule
```



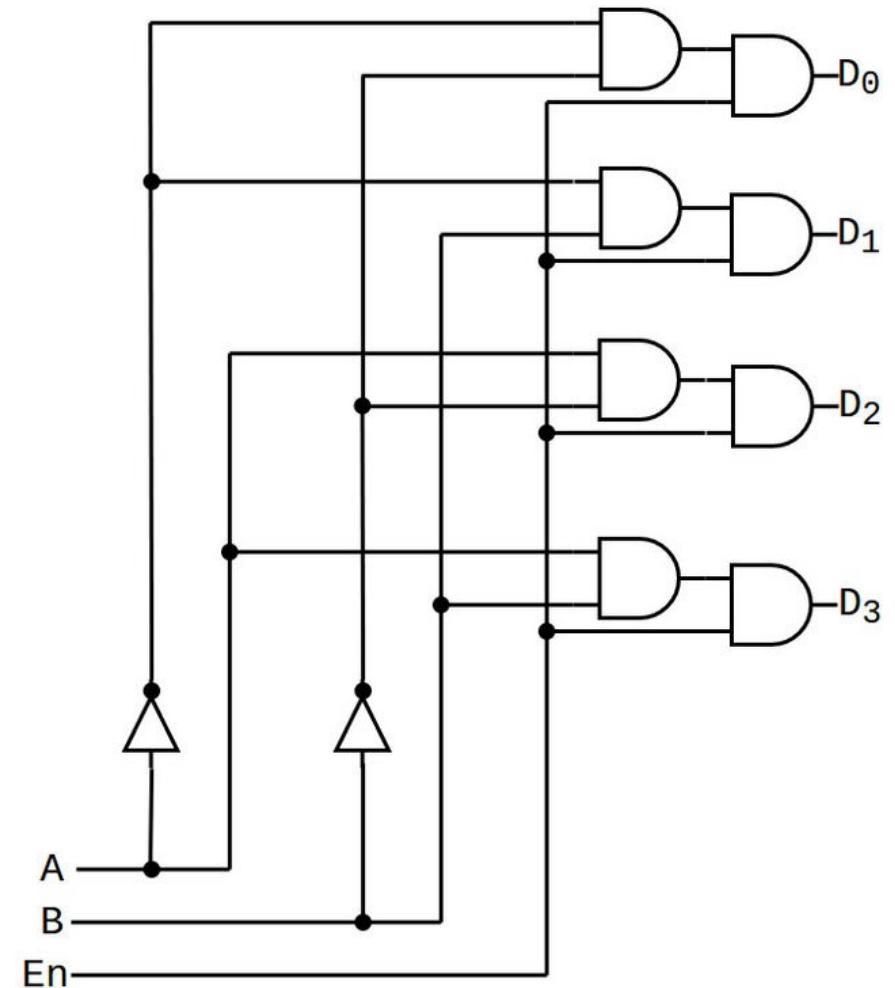
1:2 DEMUX

Decoder

- ❖ Verilog implementation of a 2-to-4 decoder using gate-level modeling
- ❖ In this model, we will explicitly use basic logic gates (AND, NOT) to construct the decoder

•Code Example (Behavioral):

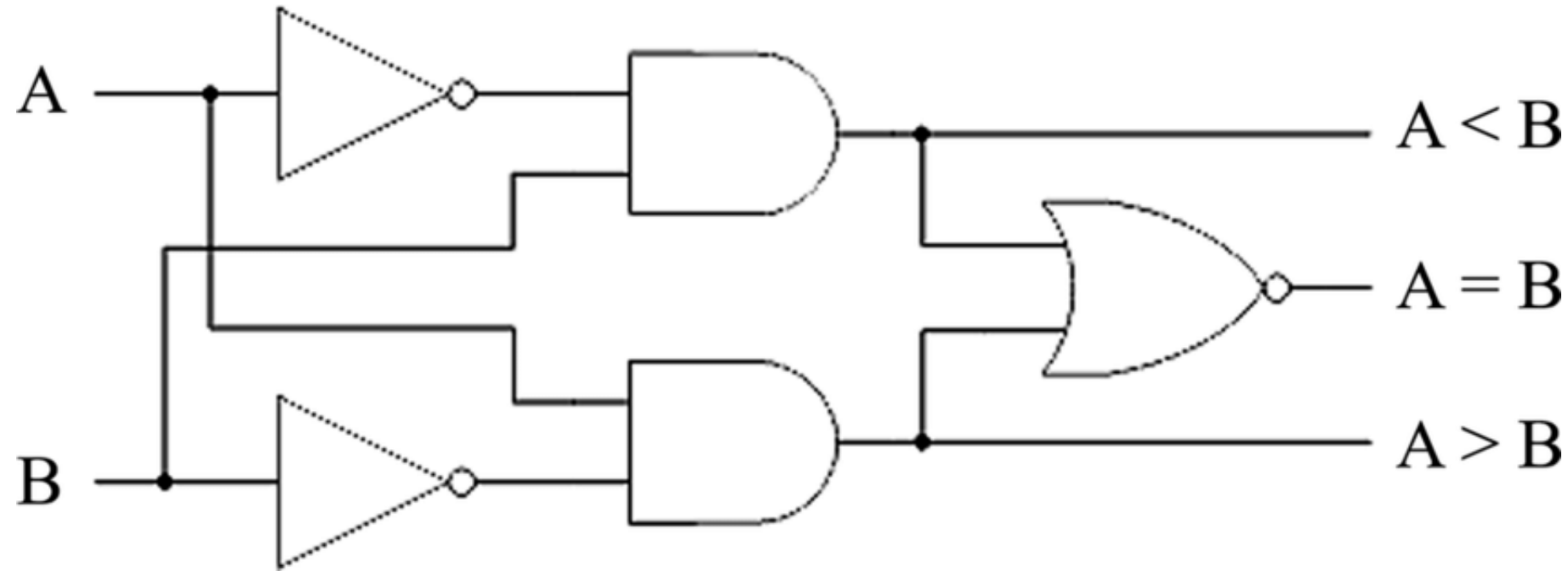
```
module decoder_2to4 (  
    input wire [1:0] A,    // 2-bit input  
    output reg [3:0] Y     // 4-bit output  
);  
    always @(*) begin  
        case (A)  
            2'b00: Y = 4'b0001; // Output D0  
            2'b01: Y = 4'b0010; // Output D1  
            2'b10: Y = 4'b0100; // Output D2  
            2'b11: Y = 4'b1000; // Output D3  
            default: Y = 4'b0000; // Default case  
        endcase  
    end  
endmodule
```



Comparator in Verilog

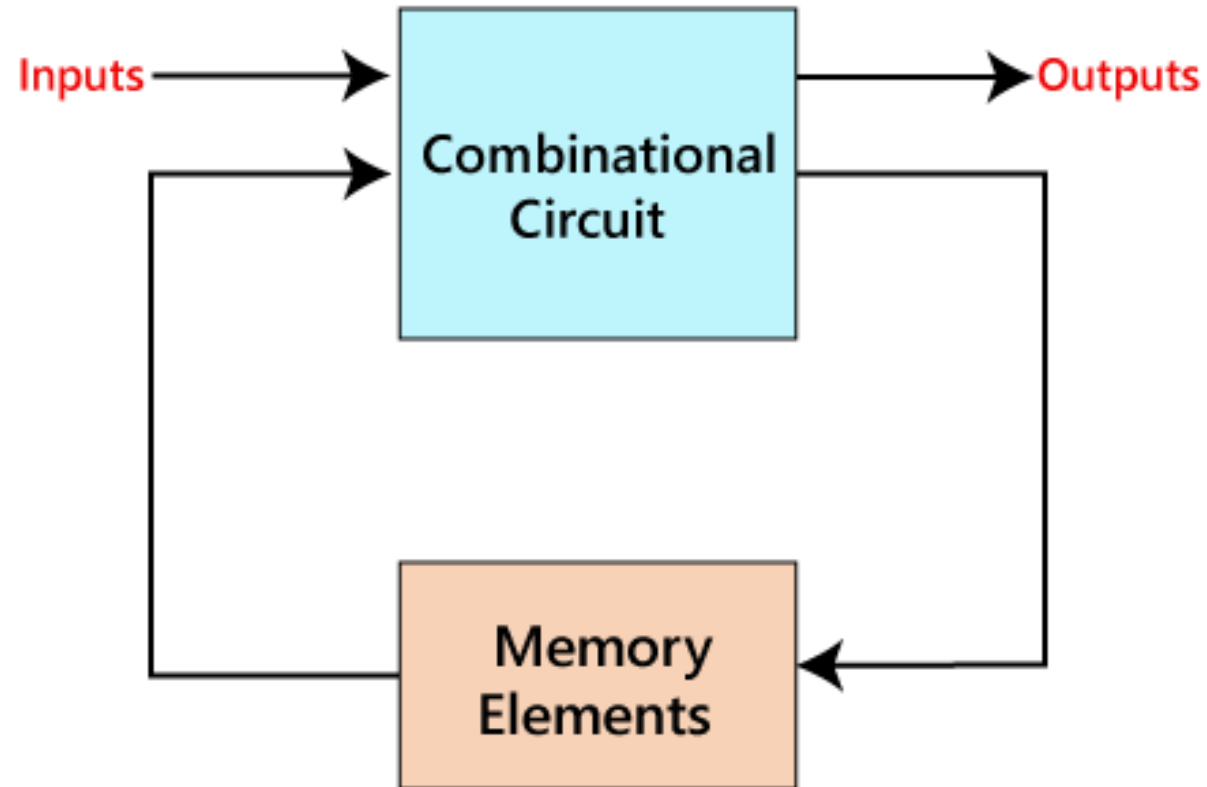
Code Example (Behavioral)

```
module comparator(
    input [3:0] A,
    input [3:0] B,
    output reg eq,
    output reg gt,
    output reg lt
);
    always @(*) begin
        eq = (A == B);
        gt = (A > B);
        lt = (A < B);
    end
endmodule
```



Sequential Circuits

- ❖ Sequential circuits are digital circuits where the output depends not only on the current inputs but also on the past inputs
- ❖ It contains memory elements that store information about previous states



Types of Sequential Circuits

❑ **Asynchronous Sequential Circuits:** These circuits can change their state at any time, depending on the inputs, without a clock signal

❑ They are generally faster but more complex to design

Examples: Latches

Synchronous Sequential Circuits: These circuits change their state at specific intervals, typically synchronized with a clock signal

Examples: Flip-flops, Counters, Registers

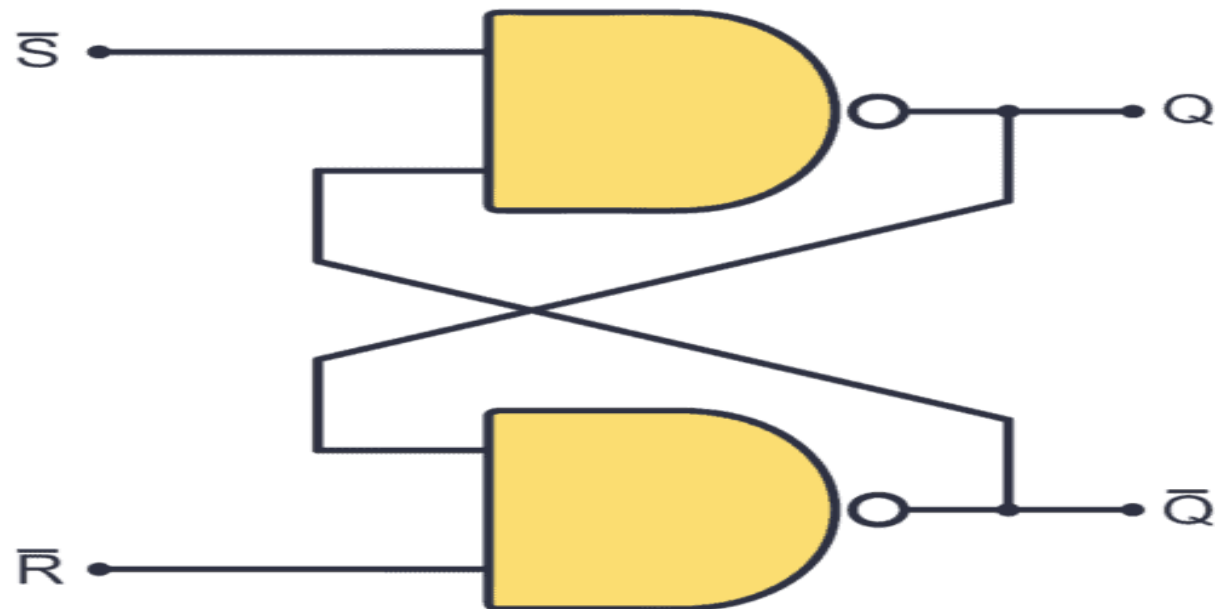
SR-Latches in Verilog

•Code Example (Behavioural)

```
module sr_latch (  
    input wire S,    // Set input  
    input wire R,    // Reset input  
    output reg Q,    // Output Q  
    output reg Qn    // Inverted output Q  
);  
  
always @ (S or R) begin  
    if (S && !R) begin  
        Q <= 1;  
        Qn <= 0;  
    end  
    else if (!S && R) begin  
        Q <= 0;  
        QN <= 1;  
    end  
    // Maintain current state if S = 0 and R = 0  
end  
  
endmodule
```

Asynchronous sequential circuit

- A latch is a basic memory device that can store one bit of information
- It is level-sensitive, which captures and holds the input state as long as a control signal (enable) is active
- ❖ SR Latch: Set-Reset latch that can be set or reset based on input signals



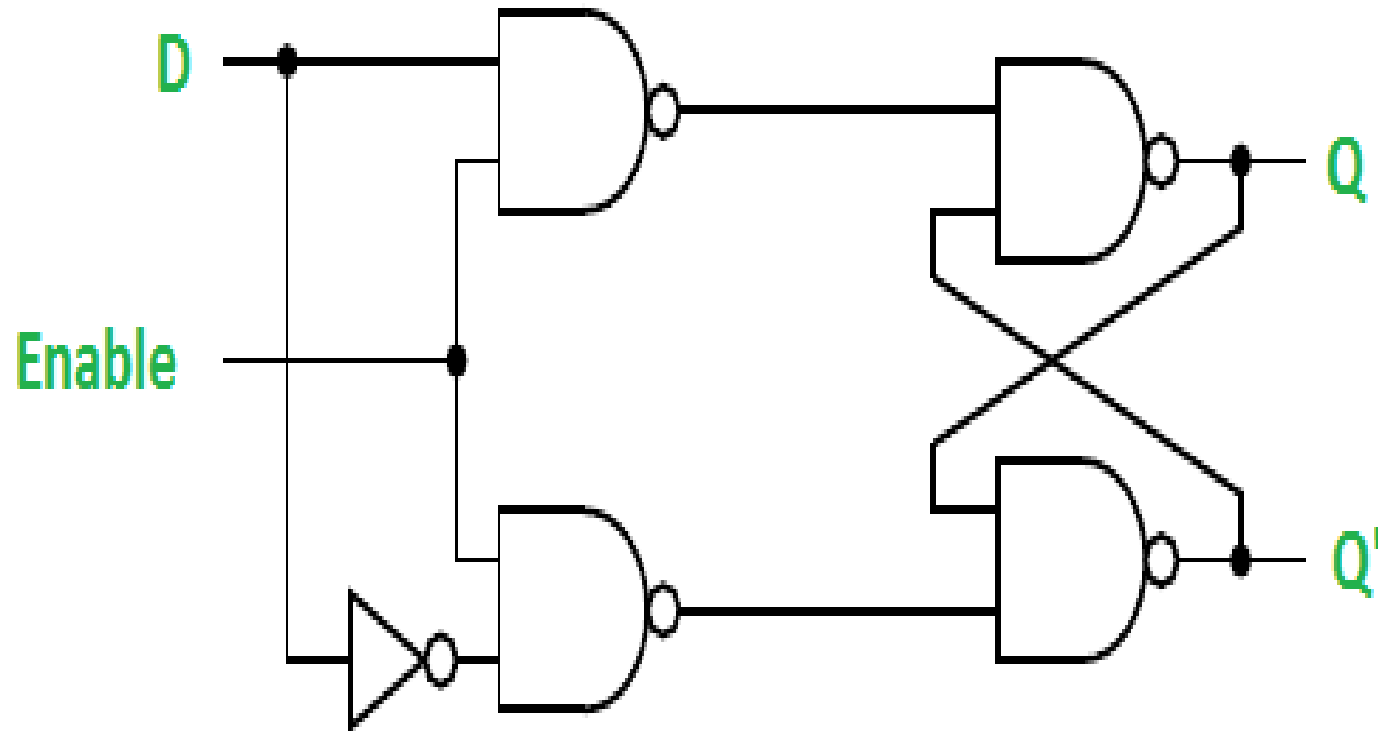
D-Latches in Verilog

Asynchronous sequential circuit

•Code Example (Behavioural)

```
module d_latch(  
    input D,  
    input Enable,  
    output reg Q  
);  
    always @(Enable or D) begin  
        if (Enable)  
            Q = D; // Latch behavior  
    end  
endmodule
```

•D Latch: Data latch that captures the input when the enable signal is high.



Flip-Flop

- A flip-flop is a digital memory circuit that can store one bit of data and is edge-sensitive
- It changes state only at specific moments, typically on the rising or falling edge of a clock signal

Types:

- **SR Flip-Flop:** Similar to the SR latch but is edge-triggered
- **D Flip-Flop:** Captures the input value on the clock edge
- **JK Flip-Flop:** A versatile flip-flop that can toggle, set, or reset based on inputs
- **T Flip-Flop:** Toggles the output state on each clock pulse when the input is high

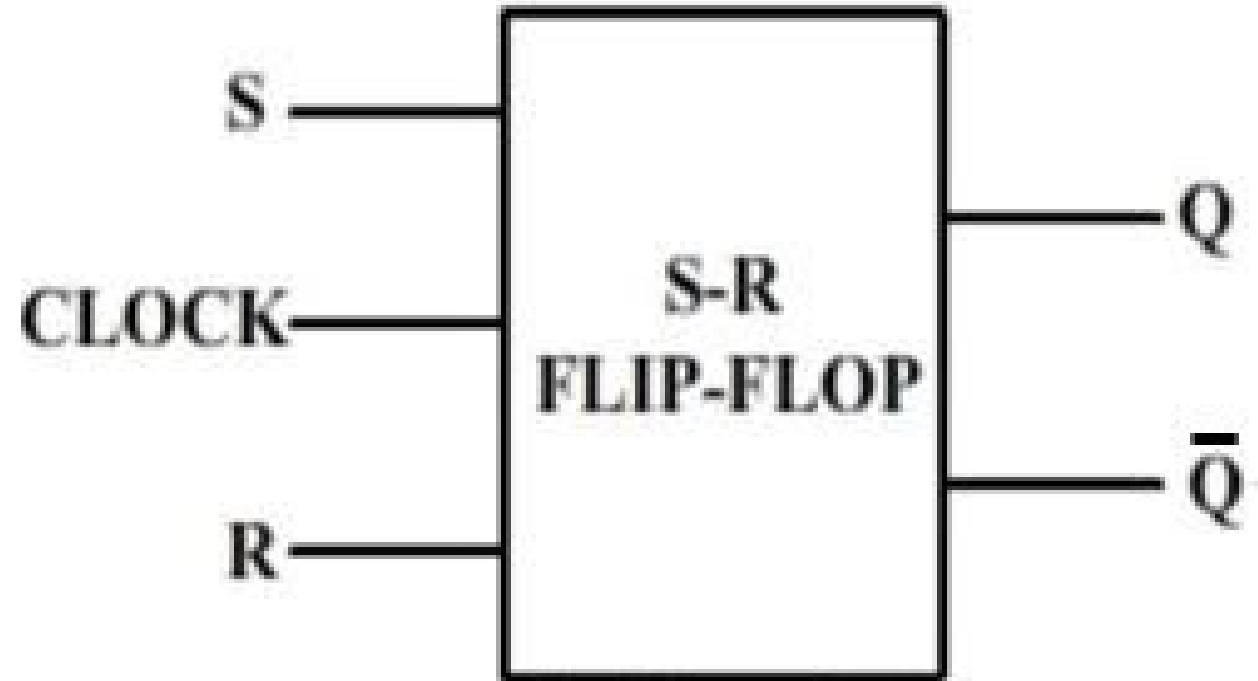
SR-Flip-Flop in Verilog

Synchronous sequential circuit

•Code Example (Behavioral)

```
module sr_flip_flop (  
    input wire S,  
    input wire R,  
    input wire clk,  
    output reg Q  
);  
always @(posedge clk) begin  
    if (S && ~R)  
        Q <= 1; // Set  
    else if (~S && R)  
        Q <= 0; // Reset  
    // No change if both are low  
end  
endmodule
```

- ❖ The SR flip-flop has two inputs: Set (S) and Reset (R)
- ❖ It sets the output (Q) to 1 when S is high and resets it to 0 when R is high
- ❖ If both inputs are low, the output remains unchanged.
- ❖ If both inputs are high, it's considered an invalid state



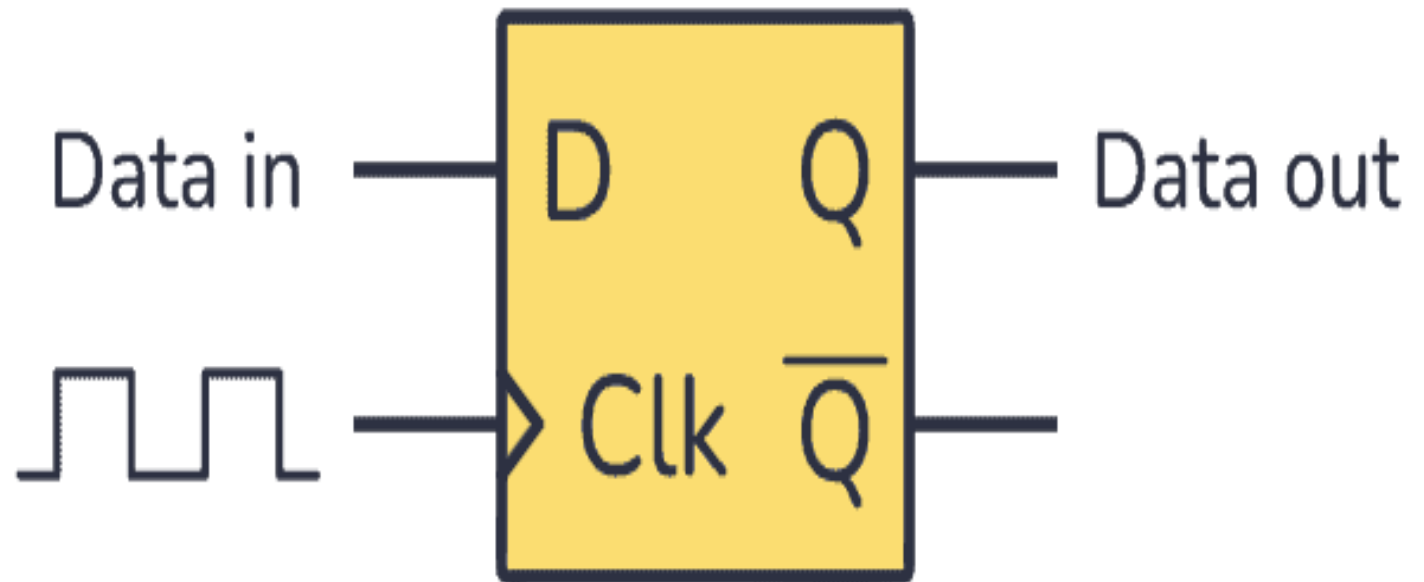
D-Flip-Flop in Verilog

Synchronous sequential circuit

•Code Example (Behavioral)

```
module d_flip_flop(  
    input D,  
    input Clk,  
    output Data_out Q  
);  
    always @(posedge clk)  
begin  
    q <= d; // D flip-flop  
behavior  
end  
endmodule
```

- ❖ The D flip-flop has a single data input (D)
- ❖ It captures the value of D at the rising edge of the clock and holds it until the next clock edge

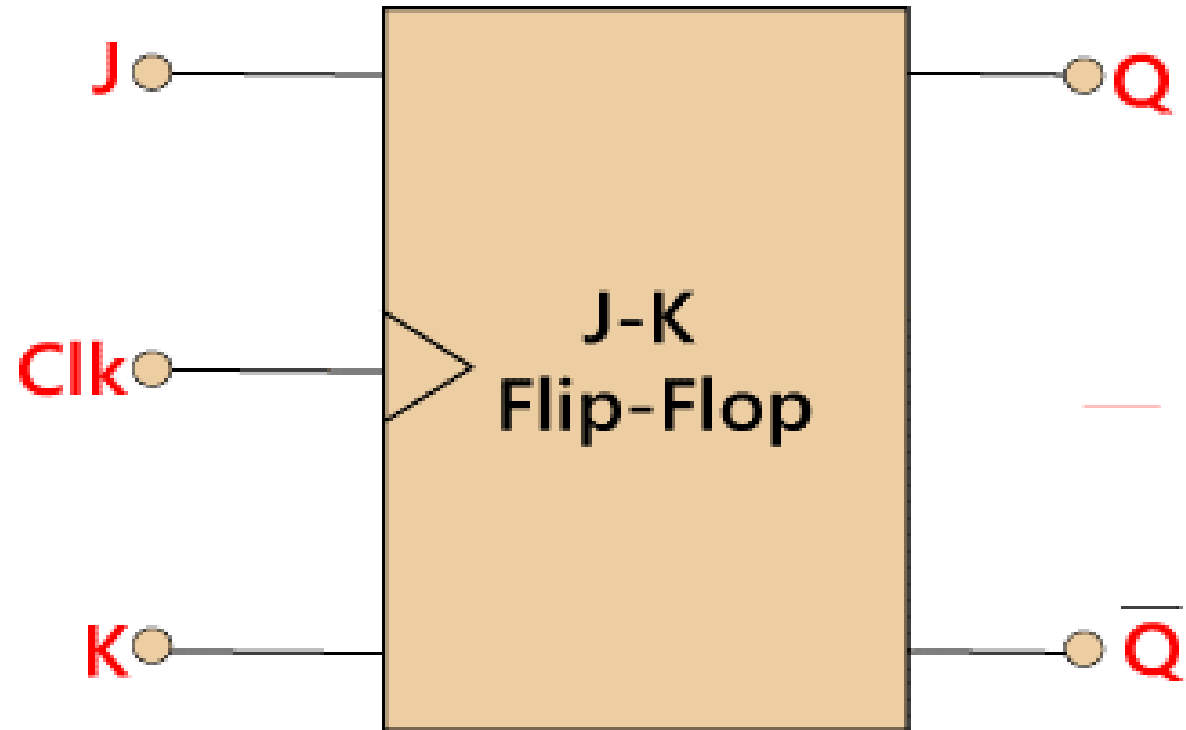


JK-Flip-Flop in Verilog

•Code Example (Behavioral)

```
module jk_flip_flop (  
    input wire J,  
    input wire K,  
    input wire clk,  
    output reg Q  
);  
always @(posedge clk) begin  
    if (J && ~K)  
        Q <= 1; // Set  
    else if (~J && K)  
        Q <= 0; // Reset  
    else if (J && K)  
        Q <= ~Q; // Toggle  
    // No change if both are low  
end  
endmodule
```

- ❖ The JK flip-flop has two inputs: J and K
- ❖ When J is high, it sets Q; when K is high, it resets Q
- ❖ If both are high, it toggles the output
- ❖ This flip-flop avoids the invalid state found in SR flip-flops

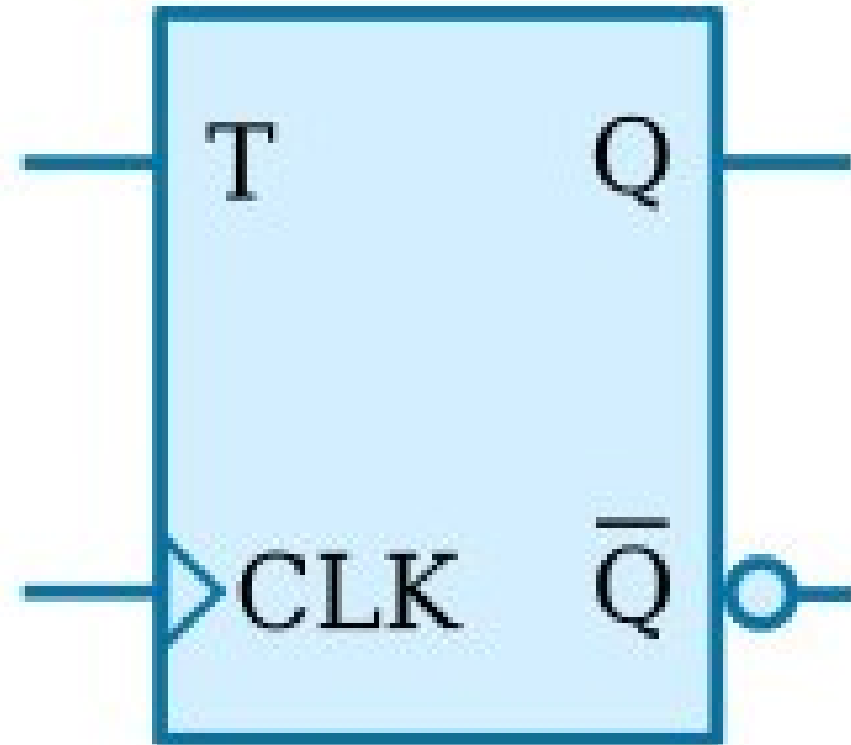


T-Flip-Flop in Verilog

•Code Example (Behavioral)

```
module t_flip_flop (  
    input wire T,  
    input wire clk,  
    output reg Q  
);  
    always @(posedge clk) begin  
        if (T)  
            Q <= ~Q; // Toggle  
        // No change if T is low  
    end  
endmodule
```

- ❖ The T flip-flop has a single input (T)
- ❖ If T is high, it toggles the output on the clock's rising edge
- ❖ If T is low, the output remains unchanged

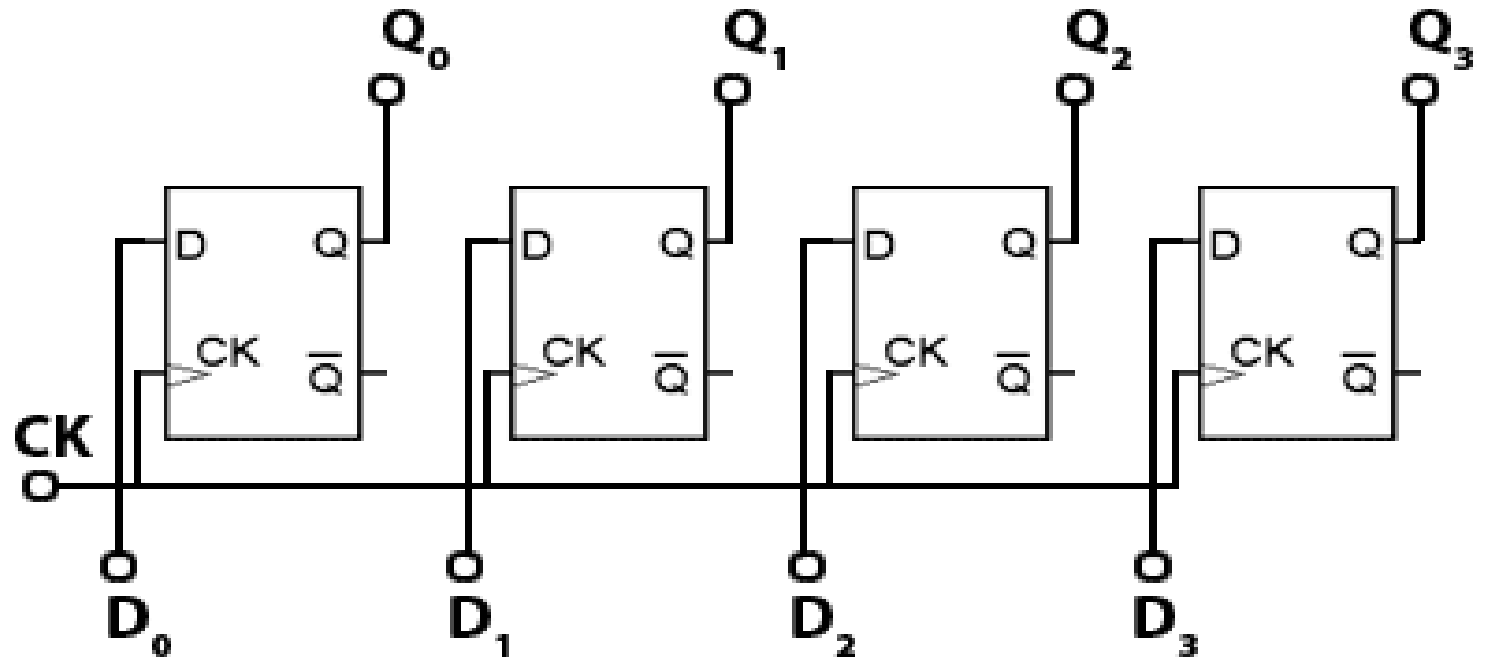


Registers in Verilog

•Code Example (Behavioral)

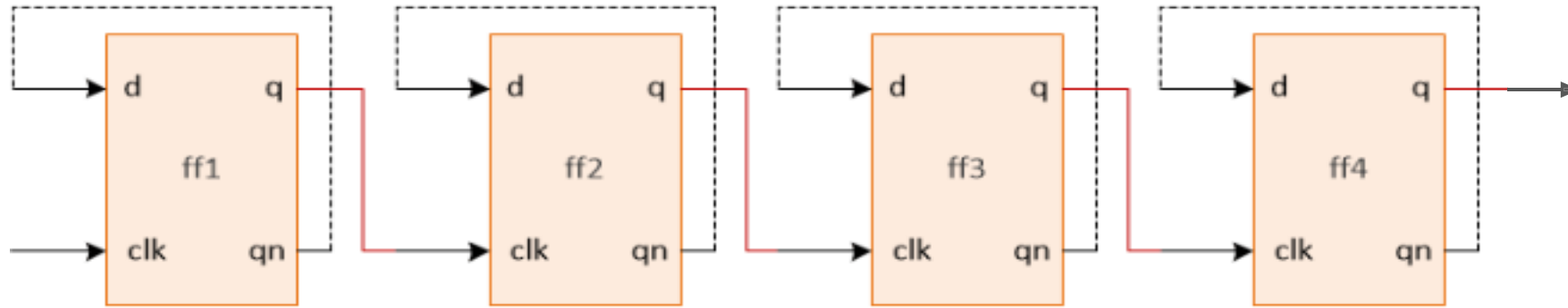
```
module register(  
    input clk,  
    input [7:0] d,  
    output reg [7:0] q  
);  
    always @(posedge clk)  
begin  
    q <= d; // Register  
behavior  
end  
endmodule
```

- ❖ A register is a set of flip-flops used to store multiple bits of data
- ❖ It acts as a temporary storage element in a digital system
- ❖ Types: Shift Registers and Universal Registers



Counter example - Using D-flip-flops

- ❑ A counter is a sequential circuit that counts pulses and can increment or decrement a value based on clock signals
- ❑ It often provides a binary output corresponding to the count
- ❑ Now, we will build a 4-bit down counter at a slightly lower level of abstraction, using D flip-flops



```
module ripple ( input clk,
                input rstn,
                output [3:0] out);

    wire q0;
    wire qn0;
    wire q1;
    wire qn1;
    wire q2;
    wire qn2;
    wire q3;
    wire qn3;
```

```
    dff dff0 ( .d (qn0),
               .clk (clk),
               .rstn (rstn),
               .q (q0),
               .qn (qn0));

    dff dff1 ( .d (qn1),
               .clk (q0),
               .rstn (rstn),
               .q (q1),
               .qn (qn1));
```

```
    dff dff2 ( .d (qn2),
               .clk (q1),
               .rstn (rstn),
               .q (q2),
               .qn (qn2));

    dff dff3 ( .d (qn3),
               .clk (q2),
               .rstn (rstn),
               .q (q3),
               .qn (qn3));

    assign out = {qn3, qn2, qn1, qn0};

endmodule
```


Learning Outcomes?

- ❖ Introduction and Components of Verilog Code_ Hardware Language
- ❖ Verilog Introduction and its types of modeling style,
- ❖ Verilog implementation of adder, subtractor, mux-demux, decoders, comparators
- ❖ Verilog Implementation of Flip Flops, latches, Registers, Counters

Thank you !